

OWASP ZAP WEB VULNERABILITY SCANNER

Complete User & Technical Manual

1. Introduction

Web applications are widely used today for education, banking, shopping, and communication. Because they store sensitive data, they are often targeted by attackers. To protect these applications, **security testing** is required.

This project implements a **Web Vulnerability Scanner** using **OWASP ZAP**, controlled through **Python and Flask**, with a **web-based interface** for easy use.

2. What is OWASP ZAP?

OWASP ZAP (Zed Attack Proxy) is an open-source security testing tool developed by **OWASP (Open Web Application Security Project)**.

Purpose of OWASP ZAP

- Identify security vulnerabilities
- Perform ethical hacking tests
- Help developers fix weaknesses
- Improve overall web security

Why ZAP?

- Free & open source
- Industry-standard tool

- Used by professionals
 - Supports automation via API
-

3. What This Project Does

This system allows users to:

- Enter a website URL
- Run a **FAST or FULL security scan**
- See **live scan progress**
- View detected vulnerabilities
- Learn about security risks

Target Users

- Students
 - Cybersecurity learners
 - Developers
 - Ethical hackers (educational use)
-

4. System Architecture Overview

Components Used

Component	Purpose
OWASP ZAP	Core vulnerability scanner
Python	Backend logic

Flask	Web server & API
HTML/CSS	Frontend interface
JavaScript	UI interaction
ZAP API	Communication bridge

Architecture Diagram (Conceptual)



5. How OWASP ZAP Works (Core Concept)

OWASP ZAP acts as a **proxy** between your system and the target website.

Proxy Concept

- All web traffic passes through ZAP
 - ZAP analyzes requests & responses
 - Vulnerabilities are detected automatically
-

6. Scan Types Explained

6.1 Spider Scan (Crawling)

Spidering means:

- Discovering all pages
- Finding URLs, forms, inputs
- Mapping the website structure

Example:

```
/login  
/register  
/products?id=1  
/admin
```

This is the **first step** in every scan.

6.2 Passive Scan (Safe Scan)

Passive scanning:

- Observes traffic
- No attacks performed
- No modification of requests

Detects:

- Missing security headers
- Cookie issues
- Information leakage
- Weak configurations

✅ Safe for production (with permission)

6.3 Active Scan (Attack Scan)

Active scanning:

- Sends attack payloads
- Tests SQL Injection, XSS, File Inclusion
- Modifies requests intentionally

 Only allowed on test systems or with permission

7. FAST vs FULL Scan Modes

Mode	Description	Speed	Risk
FAST	Spider + Passive	Fast	Very Safe
FULL	Spider + Passive + Active	Slow	Aggressive

8. Workflow of the System

Step-by-Step Process

1. User enters URL
2. Frontend sends request to Flask API
3. Flask validates URL
4. Flask calls OWASP ZAP API
5. ZAP starts spider scan
6. Passive scan runs automatically
7. (Optional) Active scan starts

8. Progress updates sent to frontend
9. Results displayed in UI

9. Live Progress System

To avoid freezing:

- Scan runs in steps
- Progress percentages are returned
- UI updates using polling

Progress Indicators

- Spider Progress (%)
- Active Scan Progress (%)

10. Understanding ZAP Logs

INFO Logs

`ZAP is now listening on localhost:8080`

✓ ZAP is running correctly

WARN Logs

`Host could not be reliably evaluated`

- ✓ Invalid or malformed URLs
- ✓ Not a system error

ERROR Logs

OutOfMemoryError: Java heap space

- ✗ ZAP ran out of memory
 - ✓ Fixed by increasing heap size
-

11. Memory Management in ZAP

ZAP is a Java application and uses **heap memory**.

Solution Used

-Xmx2048m

This allocates **2GB RAM** to ZAP.

Spider Optimization

- Reduced depth
 - Limited threads
 - Faster scanning
-

12. Understanding Vulnerability Alerts

Each alert contains:

Field	Meaning
Alert Name	Vulnerability type
Risk Level	Low / Medium / High
Description	Explanation

Solution How to fix it

Example

X-Frame-Options Header Missing

Risk: Medium

Meaning:

Website is vulnerable to clickjacking

13. Sample Test Websites (Legal)

You may test ONLY these sites:

- <http://testphp.vulnweb.com>
- <http://demo.testfire.net>
- <https://juice-shop.herokuapp.com>
- DVWA (local installation)

 **Never scan random websites**

14. Why University Websites Should Not Be Scanned

- Considered unauthorized testing
- May violate cyber laws
- Could lead to legal action

- ✓ Only scan with written permission
 - ✓ Use FAST mode only
-

15. Common Errors & Solutions

Error: Scan stuck

- ✓ Large website
 - ✓ Solution: Reduce spider depth
-

Error: Generator not JSON serializable

- ✓ Flask does not support generators
 - ✓ Solution: Use polling / convert to dict
-

Error: ZAP not responding

- ✓ ZAP daemon not running
 - ✓ Solution: Start ZAP first
-

16. Security & Ethical Considerations

This tool is for:

- Education
- Learning cybersecurity
- Ethical hacking

Not for:

- Attacking websites
 - Unauthorized scanning
 - Data theft
-

17. Learning Outcomes

By completing this project, you learned:

- Web application security
 - Ethical hacking tools
 - Backend-frontend integration
 - Proxy-based scanning
 - Real-time system design
-

18. Future Enhancements

Possible improvements:

- Authentication scanning
 - Login form testing
 - PDF vulnerability reports
 - Scan history
 - CVSS scoring
 - User authentication
-

19. Conclusion

This project demonstrates a **professional-grade web vulnerability scanner** using OWASP ZAP, Python, and Flask. It provides real-time scanning, ethical security testing, and a user-friendly interface.